IEX EDA - Carbon Trading & Energy Market Analysis

This notebook integrates MySQL + Python to perform business-level ESG analytics.

Objectives:

- Identify emission-heavy industries
- Evaluate compliance risk
- Analyze carbon price impact
- Study optimization savings
- Understand trading patterns

In [1]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sqlalchemy import create_engine
```

In [2]:
```
# Database connection parameters

username = "root"
password = "root"
host = "localhost"
port = "3306"
database = "Carbon_Trading_DB"

engine = create_engine(f"mysql+pymysql://{username}:{password}@{host}:{port}/{database}")

engine.connect()
```

Out[2]:
```
<sqlalchemy.engine.base.Connection at 0x19f526870d0>
```

In [3]:
```
df = pd.read_sql("SELECT * FROM carbon_data", engine)
df.head()
```

Out[3]:

	Company_ID	Industry_Type	Date	Energy_Demand_MWh	Fuel_Type	Emission_Produced_tCO2	Emission_Allowance_tCO2	Car
0	C099	Manufacturing	2025-03-20	2523.15	Renewable	1532.65	1428.65	
1	C015	Cement	2024-01-12	NaN	Renewable	1205.54	1228.54	
2	C047	Manufacturing	2024-08-28	977.27	Renewable	731.40	721.40	
3	C090	nan	2024-01-11	NaN	Natural Gas	1396.17	1258.17	
4	C095	Energy	2024-11-03	NaN	MIXED FUEL	1485.77	1533.77	

In [4]:
```
from sqlalchemy import text

create_view_query = """
CREATE OR REPLACE VIEW vw_carbon_clean AS
SELECT
    Company_ID,

    -- Date Standardization
    STR_TO_DATE(Date, '%Y-%m-%d') AS Date,

    -- Industry Cleaning
    CASE
        WHEN LOWER(TRIM(Industry_Type)) IN ('nan','none','unknown','null','')
        THEN 'Manufacturing'
        ELSE CONCAT(
            UPPER(LEFT(TRIM(Industry_Type),1)),
            LOWER(SUBSTRING(TRIM(Industry_Type),2))
        )
    END AS Industry_Type,

    -- Fuel Type Cleaning
    CONCAT(
        UPPER(LEFT(TRIM(Fuel_Type),1)),
        LOWER(SUBSTRING(TRIM(Fuel_Type),2))
    ) AS Fuel_Type,
```

```
    -- Numeric Handling
    COALESCE(Energy_Demand_MWh,0) AS Energy_Demand_MWh,
    COALESCE(Emission_Produced_tCO2,0) AS Emission_Produced_tCO2,
    COALESCE(Emission_Allowance_tCO2,0) AS Emission_Allowance_tCO2,
    COALESCE(Carbon_Price_USD_per_t,0) AS Carbon_Price_USD_per_t,
    COALESCE(Credits_Traded_tCO2,0) AS Credits_Traded_tCO2,
    COALESCE(Compliance_Cost_USD,0) AS Compliance_Cost_USD,
    COALESCE(Carbon_Cost_Savings_USD,0) AS Carbon_Cost_Savings_USD,

    Verification_Status,
    Optimization_Scenario,
    Transaction_Type,

    -- Derived Column
    COALESCE(Emission_Allowance_tCO2,0)
    - COALESCE(Emission_Produced_tCO2,0) AS Emission_Gap

FROM carbon_data;
"""

with engine.connect() as conn:
    conn.execute(text(create_view_query))
    conn.commit()
```

Python Data Cleaning and Preprocessing

In [5]:

```
# ---------------------------------------------------
# 1 Standardize column names (Title Case)
# ---------------------------------------------------
df.columns = (
    df.columns
        .str.strip()
        .str.lower()
        .str.replace(" ", "_")
        .str.title()
)

# ---------------------------------------------------
# 2 Convert Date column
# ---------------------------------------------------
df["Date"] = pd.to_datetime(df["Date"], errors="coerce")

# ---------------------------------------------------
# 3 Fix numeric columns stored as object
# ---------------------------------------------------
numeric_cols = [
    "Emission_Allowance_Tco2",
    "Emission_Produced_Tco2",
    "Energy_Demand_Mwh",
    "Carbon_Price_Usd_Per_T",
]

for col in numeric_cols:
    df[col] = pd.to_numeric(df[col], errors="coerce")

# ---------------------------------------------------
# 4 Create Emission Gap
# ---------------------------------------------------
df["Emission_Gap"] = (
    df["Emission_Allowance_Tco2"]
    - df["Emission_Produced_Tco2"]
)

# ---------------------------------------------------
# 5 Clean categorical columns
# ---------------------------------------------------
# ---------------------------------------------------
# 5 Clean categorical columns (PROPER STANDARDIZATION)
# ---------------------------------------------------
obj_cols = df.select_dtypes(include="object").columns

for col in obj_cols:


    df[col] = df[col].astype(str).str.strip()

    df[col] = df[col].str.lower()

    df[col] = df[col].replace(
        ["nan", "none", "unknown", "null", ""],
        np.nan
    )
```

```
    if df[col].isna().any():
        df[col] = df[col].fillna(df[col].mode()[0])

    df[col] = df[col].str.title()

# ---------------------------------------------------
# 6 Handle numeric missing values
# ---------------------------------------------------
num_cols = df.select_dtypes(include=["number"]).columns

for col in num_cols:
    df[col] = df[col].replace([np.inf, -np.inf], np.nan)
    df[col] = df[col].fillna(df[col].median())

# ---------------------------------------------------
# 7 Final Check
# ---------------------------------------------------
df.info()
df.isnull().sum()
df.head()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 4511 entries, 0 to 4510
Data columns (total 15 columns):
 #   Column                   Non-Null Count  Dtype
---  ------                   --------------  -----
 0   Company_Id               4511 non-null   object
 1   Industry_Type            4511 non-null   object
 2   Date                     4511 non-null   datetime64[ns]
 3   Energy_Demand_Mwh        4511 non-null   float64
 4   Fuel_Type                4511 non-null   object
 5   Emission_Produced_Tco2   4511 non-null   float64
 6   Emission_Allowance_Tco2  4511 non-null   float64
 7   Carbon_Price_Usd_Per_T   4511 non-null   float64
 8   Credits_Traded_Tco2      4511 non-null   float64
 9   Verification_Status      4511 non-null   object
 10  Compliance_Cost_Usd      4511 non-null   float64
 11  Optimization_Scenario    4511 non-null   object
 12  Carbon_Cost_Savings_Usd  4511 non-null   float64
 13  Transaction_Type         4511 non-null   object
 14  Emission_Gap             4511 non-null   float64
dtypes: datetime64[ns](1), float64(8), object(6)
memory usage: 528.8+ KB
```

Out[5]:

	Company_Id	Industry_Type	Date	Energy_Demand_Mwh	Fuel_Type	Emission_Produced_Tco2	Emission_Allowance_Tco2	Carb
0	C099	Manufacturing	2025-03-20	2523.15	Renewable	1532.65	1428.65	
1	C015	Cement	2024-01-12	1747.79	Renewable	1205.54	1228.54	
2	C047	Manufacturing	2024-08-28	977.27	Renewable	731.40	721.40	
3	C090	Manufacturing	2024-01-11	1747.79	Natural Gas	1396.17	1258.17	
4	C095	Energy	2024-11-03	1747.79	Mixed Fuel	1485.77	1533.77	

In [6]:

```
df.columns
```

Out[6]:

```
Index(['Company_Id', 'Industry_Type', 'Date', 'Energy_Demand_Mwh', 'Fuel_Type',
       'Emission_Produced_Tco2', 'Emission_Allowance_Tco2',
       'Carbon_Price_Usd_Per_T', 'Credits_Traded_Tco2', 'Verification_Status',
       'Compliance_Cost_Usd', 'Optimization_Scenario',
       'Carbon_Cost_Savings_Usd', 'Transaction_Type', 'Emission_Gap'],
      dtype='object')
```

Loadinng Data for SQL EDA

In [7]:

```
df.to_sql(
    name="sql_eda_carbon_clean",   # change name
    con=engine,
    if_exists="replace",
    index=False,
    method="multi"
)

print("DataFrame successfully saved ✅")
```

```
DataFrame successfully saved ✅
```

In [8]:
```
%load_ext sql
%sql mysql+pymysql://root:root@localhost:3306/Carbon_Trading_DB
```

In [9]:
```
%%sql
SHOW TABLES;
```

```
 * mysql+pymysql://root:***@localhost:3306/Carbon_Trading_DB
5 rows affected.
```

Out[9]:

Tables_in_carbon_trading_db
carbon_data
sql_eda_carbon_clean
sql_eda_vw_carbon_clean
vw_carbon_clean
vw_carbon_data

In [10]:
```
%%sql
SELECT * FROM sql_eda_carbon_clean LIMIT 5;
```

```
 * mysql+pymysql://root:***@localhost:3306/Carbon_Trading_DB
5 rows affected.
```

Out[10]:

Company_Id	Industry_Type	Date	Energy_Demand_Mwh	Fuel_Type	Emission_Produced_Tco2	Emission_Allowance_Tco2	Carb
C099	Manufacturing	2025-03-20 00:00:00	2523.15	Renewable	1532.65	1428.65	
C015	Cement	2024-01-12 00:00:00	1747.79	Renewable	1205.54	1228.54	
C047	Manufacturing	2024-08-28 00:00:00	977.27	Renewable	731.4	721.4	
C090	Manufacturing	2024-01-11 00:00:00	1747.79	Natural Gas	1396.17	1258.17	
C095	Energy	2024-11-03 00:00:00	1747.79	Mixed Fuel	1485.77	1533.77	

In [11]:
```
# Helper function to add labels on top of bars

def add_labels(ax, fmt="{:,.0f}", rotation=0):

    for p in ax.patches:
        height = p.get_height()
        ax.annotate(
            fmt.format(height),
            (p.get_x() + p.get_width()/2., height),
            ha='center',
            va='bottom',
            fontsize=9,
            rotation=rotation,
            xytext=(0,5),
            textcoords='offset points'
        )
```

BUSINESS EDA QUESTIONS

Q1.Industry wise Emission CO2 Emission Produced.

SQL Business Query

In [12]:
```
%%sql
SELECT Industry_Type,
       round(SUM(Emission_Produced_tCO2), 2) AS Total_Emission
FROM sql_eda_carbon_clean
GROUP BY Industry_Type
ORDER BY Total_Emission DESC;
```

```
 * mysql+pymysql://root:***@localhost:3306/Carbon_Trading_DB
4 rows affected.
```

Out[12]:

Industry_Type	Total_Emission
Manufacturing	1294280.87
Cement	1028122.42
Steel	1013428.36
Energy	999452.12

Python Business Query

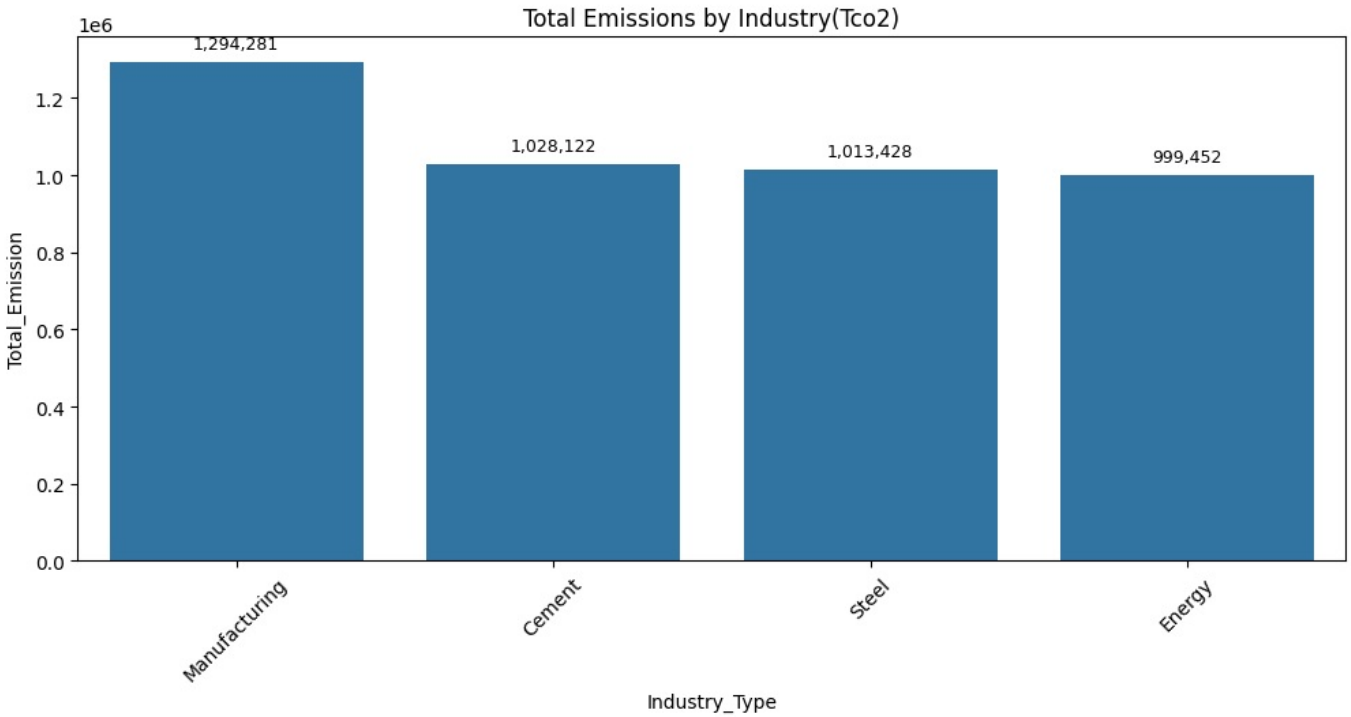
In [13]:
```
q1 = (
    df.groupby("Industry_Type")["Emission_Produced_Tco2"]
      .sum()
      .reset_index(name="Total_Emission")
      .sort_values(by="Total_Emission", ascending=False)
)


q1
```

Out[13]:

	Industry_Type	Total_Emission
2	Manufacturing	1294280.87
0	Cement	1028122.42
3	Steel	1013428.36
1	Energy	999452.12

Visualization of Query

In [14]:
```
plt.figure(figsize=(12,5))
ax = sns.barplot(data=q1, x="Industry_Type", y="Total_Emission")
add_labels(ax)
plt.xticks(rotation=45)
plt.title("Total Emissions by Industry(Tco2)")
plt.show()
```



Q2 — Which companies consume the most energy on average?

SQL Business Query

In [15]:
```
%%sql
SELECT
    Company_ID,
    ROUND(AVG(Energy_Demand_MWh), 2) AS Avg_Energy_Demand_MWh
FROM sql_eda_carbon_clean
GROUP BY Company_ID
ORDER BY Avg_Energy_Demand_MWh DESC
LIMIT 5;
```

```
 * mysql+pymysql://root:***@localhost:3306/Carbon_Trading_DB
5 rows affected.
```

Out[15]:

Company_ID	Avg_Energy_Demand_MWh
C007	1962.78
C029	1950.67
C043	1948.38
C005	1944.7
C045	1936.61

Python Business Query

In [16]:

```
q_energy_company = (
    df.groupby("Company_Id")["Energy_Demand_Mwh"]
      .mean()
      .reset_index(name="Avg_Energy_Demand(MWh)")
      .sort_values(by="Avg_Energy_Demand(MWh)", ascending=False)
)

q_energy_company.round(2).head()
```

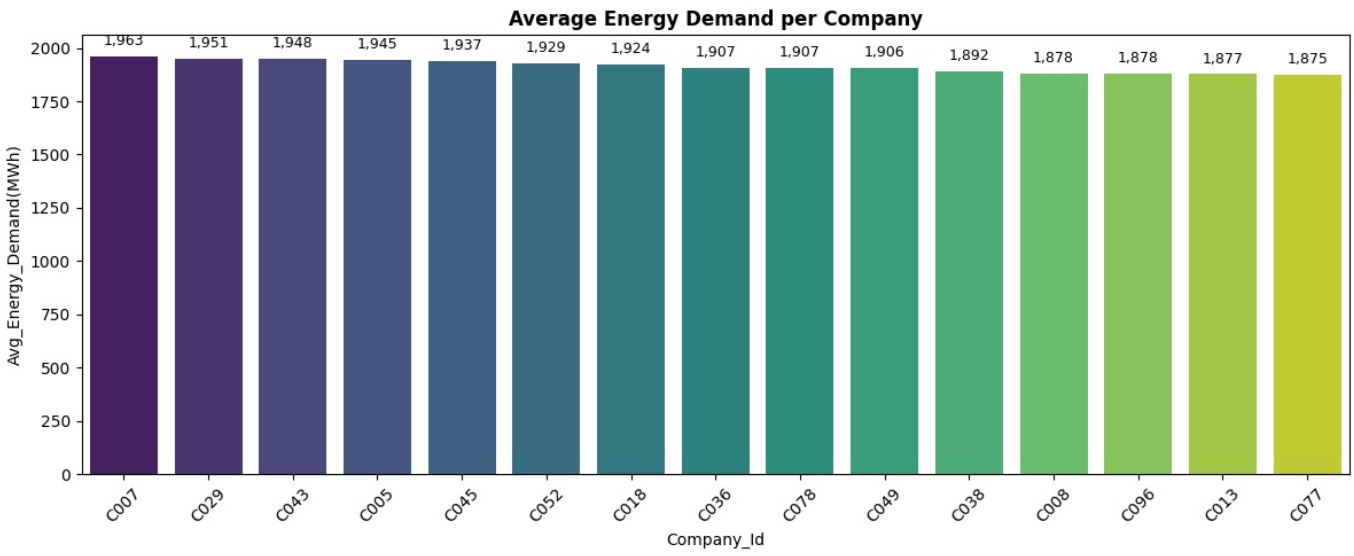
Out[16]:

	Company_Id	Avg_Energy_Demand(MWh)
6	C007	1962.78
28	C029	1950.67
42	C043	1948.38
4	C005	1944.70
44	C045	1936.61

Visualization of Query

In [17]:

```
plt.figure(figsize=(12,5))

ax = sns.barplot(
    data=q_energy_company.head(15),
    x="Company_Id",
    y="Avg_Energy_Demand(MWh)",
    hue="Company_Id",       # assign hue
    palette="viridis",
    legend=False            # remove legend
)

add_labels(ax)

plt.xticks(rotation=45)
plt.title("Average Energy Demand per Company", weight="bold")
plt.tight_layout()
plt.show()
```



Q3 — Emission Gap(Emission Allowed - Emission Produced)

SQL Business Query

In [18]:
```
%%sql
SELECT
    Company_ID,
    SUM(Emission_Gap) AS Gap
FROM sql_eda_carbon_clean
GROUP BY Company_ID
ORDER BY Gap DESC
LIMIT 5;
```

```
 * mysql+pymysql://root:***@localhost:3306/Carbon_Trading_DB
5 rows affected.
```

Out[18]:

Company_ID	Gap
C072	2471.0
C055	2096.0
C010	1925.9999999999995
C081	1674.0
C007	1434.0

Python Business Query

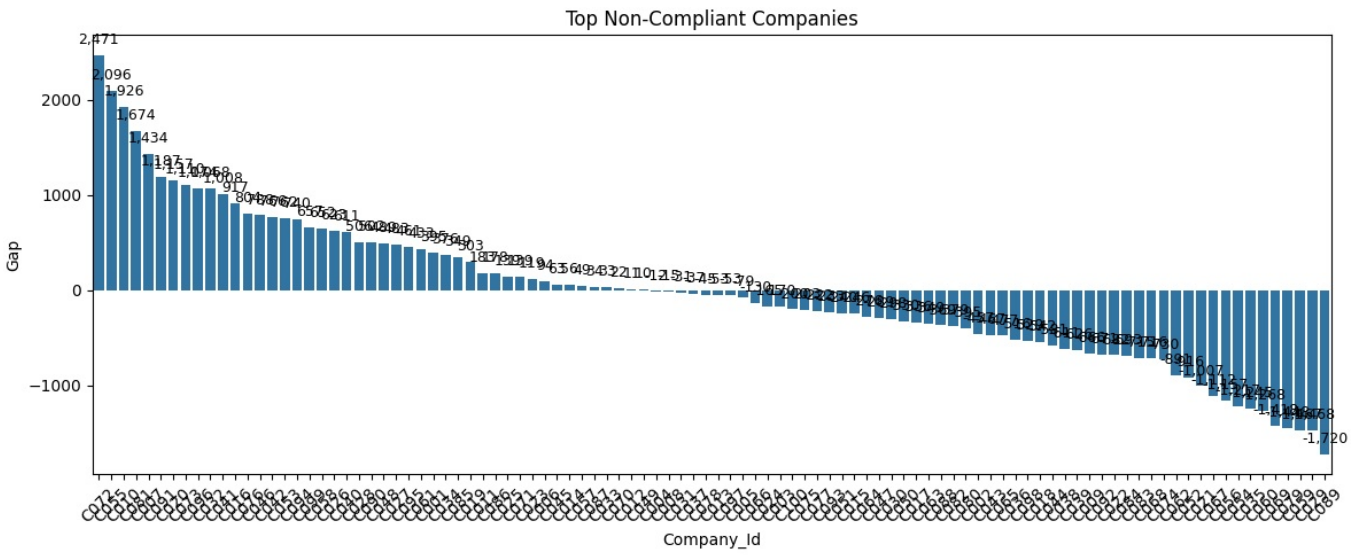
In [19]:
```
q3 = (
    df.groupby("Company_Id")["Emission_Gap"]
      .sum()
      .reset_index(name="Gap")
      .sort_values(by="Gap", ascending=False))

q3.head(10)
```

Out[19]:

	Company_Id	Gap
71	C072	2471.0
54	C055	2096.0
9	C010	1926.0
80	C081	1674.0
6	C007	1434.0
90	C091	1187.0
19	C020	1157.0
72	C073	1110.0
95	C096	1074.0
31	C032	1068.0

Visualization of Query

In [20]:
```
plt.figure(figsize=(12,5))
ax = sns.barplot(data=q3, x="Company_Id", y="Gap")
add_labels(ax)
plt.xticks(rotation=45)
plt.title("Top Non-Compliant Companies")
plt.tight_layout()
plt.show()
```



Q4 — Carbon Price vs Compliance Cost

SQL Business Query

In [21]:
```
%%sql
SELECT
    Company_ID,
    ROUND(SUM(Compliance_Cost_USD), 2) AS Total_Compliance_Cost,
    ROUND(SUM(Carbon_Cost_Savings_USD), 2) AS Total_Carbon_Savings
FROM sql_eda_carbon_clean
GROUP BY Company_ID
ORDER BY Total_Compliance_Cost DESC limit 5;
```

```
 * mysql+pymysql://root:***@localhost:3306/Carbon_Trading_DB
5 rows affected.
```

Out[21]:

Company_ID	Total_Compliance_Cost	Total_Carbon_Savings
C069	846698.04	99105.74
C035	757453.67	99237.2
C020	716680.72	84674.43
C006	712388.25	105558.21
C030	710950.89	94523.54

Python Business Query

In [22]:
```
q_cost_vs_saving = (
    df.groupby("Company_Id")
      .agg(
          Total_Compliance_Cost=("Compliance_Cost_Usd", "sum"),
          Total_Carbon_Savings=("Carbon_Cost_Savings_Usd", "sum")
      )
      .reset_index()
      .sort_values(by="Total_Compliance_Cost", ascending=False)
)

q_cost_vs_saving
```

Out[22]:

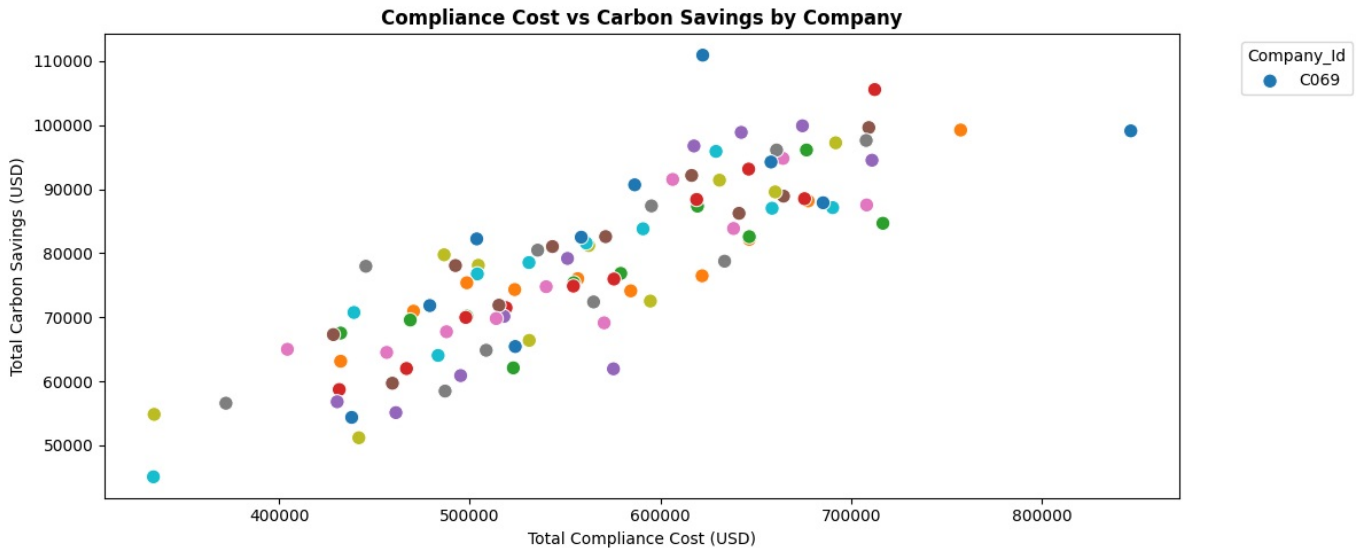
	Company_Id	Total_Compliance_Cost	Total_Carbon_Savings
68	C069	846698.04	99105.74
34	C035	757453.67	99237.20
19	C020	716680.72	84674.43
5	C006	712388.25	105558.21
29	C030	710950.89	94523.54
...	...	...	...
27	C028	428445.98	67296.70
46	C047	404341.97	64996.78
9	C010	372106.80	56580.10
38	C039	334452.65	54837.34
31	C032	334075.80	45088.00

100 rows × 3 columns

Visualization of Query

In [23]:
```
plt.figure(figsize=(12,5))

sns.scatterplot(
    data=q_cost_vs_saving,
    x="Total_Compliance_Cost",
    y="Total_Carbon_Savings",
    hue="Company_Id",
    palette="tab10",
    legend=False,
    s=80
)

plt.title("Compliance Cost vs Carbon Savings by Company", weight="bold")
plt.xlabel("Total Compliance Cost (USD)")
plt.ylabel("Total Carbon Savings (USD)")
legend_labels = q_cost_vs_saving["Company_Id"].tolist()
```

```
plt.legend(title="Company_Id", labels=legend_labels, bbox_to_anchor=(1.05 , 1), loc='upper left')

plt.tight_layout()
plt.show()
```



Q5 — Fuel Type Emission Comparison

SQL Business Query

In [24]:
```
%%sql
SELECT
    Fuel_Type,
    ROUND(SUM(Emission_Produced_tCO2), 2) AS Total_Emission
FROM sql_eda_carbon_clean
GROUP BY Fuel_Type
order by Total_Emission DESC;
```

```
 * mysql+pymysql://root:***@localhost:3306/Carbon_Trading_DB
4 rows affected.
```

Out[24]:

Fuel_Type	Total_Emission
Natural Gas	1451678.67
Coal	1164180.16
Mixed Fuel	1150557.21
Renewable	568867.73

Python Business Query

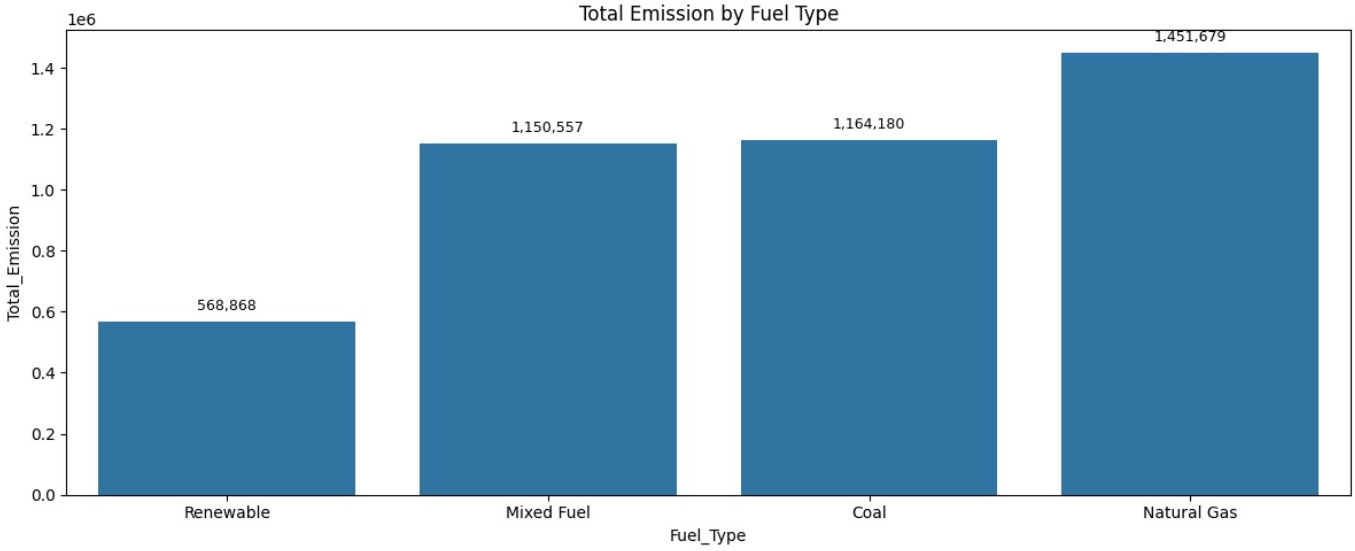
In [25]:
```
q5 = (
    df.groupby("Fuel_Type")["Emission_Produced_Tco2"]
      .sum()
      .reset_index(name="Total_Emission")
      .sort_values(by="Total_Emission", ascending=True)
)


q5.round(2)
```

Out[25]:

	Fuel_Type	Total_Emission
3	Renewable	568867.73
1	Mixed Fuel	1150557.21
0	Coal	1164180.16
2	Natural Gas	1451678.67

Visualization of Query

In [26]:
```
plt.figure(figsize=(12,5))
ax = sns.barplot(data=q5, x="Fuel_Type", y="Total_Emission")
add_labels(ax)
plt.title("Total Emission by Fuel Type")
plt.tight_layout()
plt.show()
```



Q6 — Verification Status Analysis

SQL Business Query

In [27]:
```
%%sql
SELECT
    Verification_Status,
    ROUND(SUM(Emission_Produced_tCO2), 2) AS Total_Emission
FROM sql_eda_carbon_clean
GROUP BY Verification_Status;
```

```
 * mysql+pymysql://root:***@localhost:3306/Carbon_Trading_DB
2 rows affected.
```

Out[27]:

Verification_Status	Total_Emission
Disputed	2054506.2
Verified	2280777.57

Python Business Query

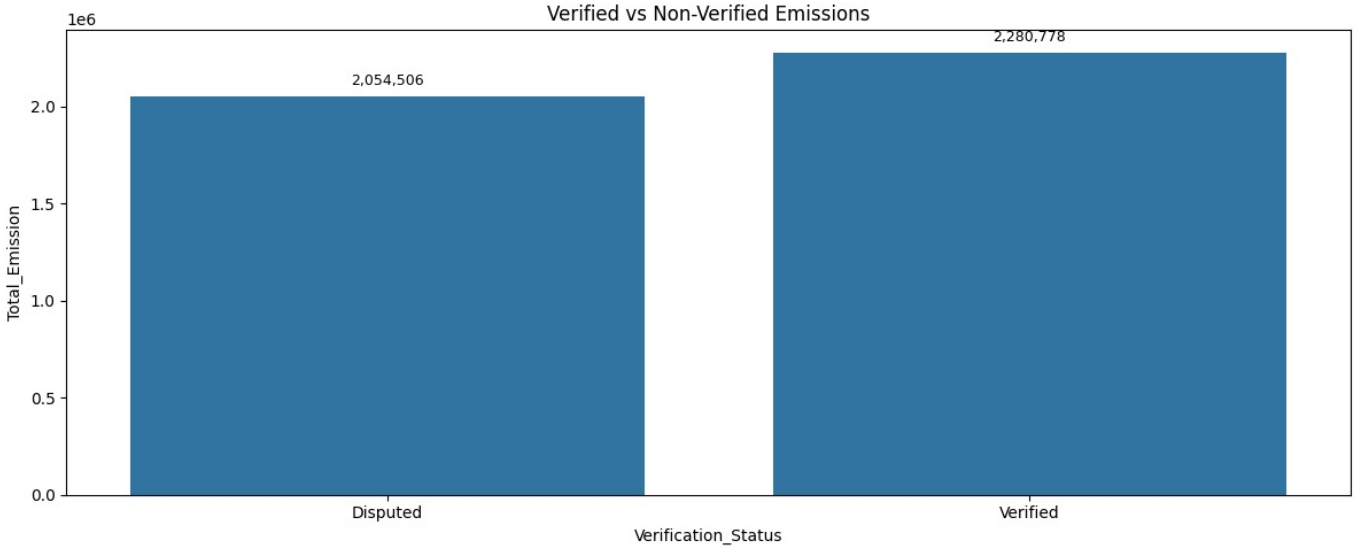
In [28]:
```
q6 = (
    df.groupby("Verification_Status")["Emission_Produced_Tco2"]
      .sum()
      .reset_index(name="Total_Emission")
)

q6
```

Out[28]:

	Verification_Status	Total_Emission
0	Disputed	2054506.20
1	Verified	2280777.57

Visualization of Query

In [29]:
```
plt.figure(figsize=(12,5))
ax = sns.barplot(data=q6, x="Verification_Status", y="Total_Emission")
add_labels(ax)
plt.title("Verified vs Non-Verified Emissions")
plt.tight_layout()
plt.show()
```



Q7 — Optimization Scenario Savings

SQL Business Query

In [30]:
```
%%sql
SELECT
    Optimization_Scenario,
    ROUND(SUM(Carbon_Cost_Savings_USD), 2) AS Savings
FROM sql_eda_carbon_clean
GROUP BY Optimization_Scenario
ORDER BY Savings DESC;
```

```
 * mysql+pymysql://root:***@localhost:3306/Carbon_Trading_DB
3 rows affected.
```

Out[30]:

Optimization_Scenario	Savings
High_Demand	2638701.26
Low_Demand	2629769.9
Price_Surge	2565525.63

Python Business Query

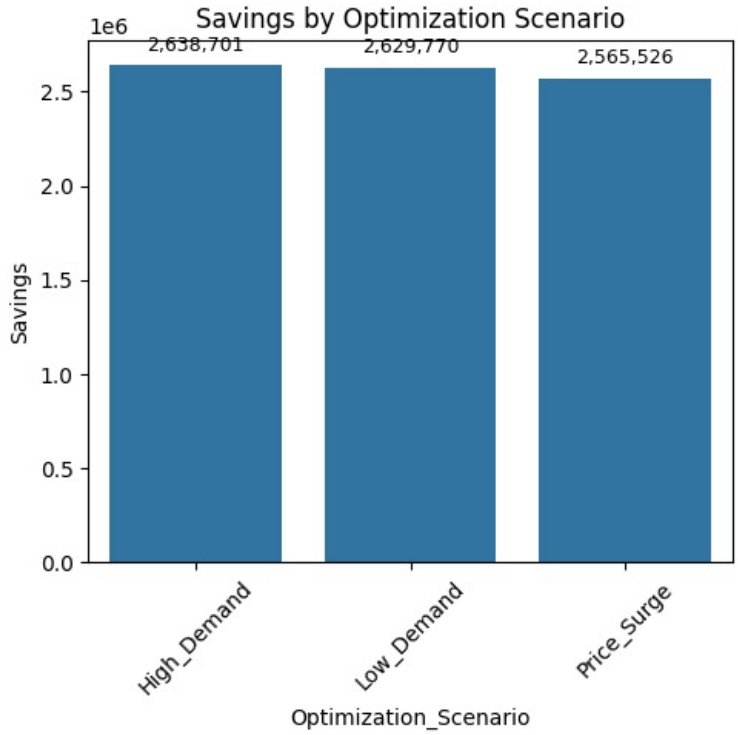
In [31]:
```
q7 = (
    df.groupby("Optimization_Scenario")["Carbon_Cost_Savings_Usd"]
      .sum()
      .reset_index(name="Savings")
)

q7
```

Out[31]:

	Optimization_Scenario	Savings
0	High_Demand	2638701.26
1	Low_Demand	2629769.90
2	Price_Surge	2565525.63

Visualization of Query

In [32]:
```
plt.figure(figsize=(5,5))
ax = sns.barplot(data=q7, x="Optimization_Scenario", y="Savings")
add_labels(ax)
plt.xticks(rotation=45)
plt.title("Savings by Optimization Scenario")
plt.tight_layout()
plt.show()
```



Q8 — How do Buy, Sell, and Hold transactions vary quarterly?

SQL Business Query

In [33]:
```
%%sql
SELECT
    CONCAT(YEAR(Date), '-Q', QUARTER(Date)) AS Quarter,
    Transaction_Type,
    COUNT(*) AS Count
FROM sql_eda_carbon_clean
GROUP BY Quarter, Transaction_Type
ORDER BY Quarter, Transaction_Type
limit 5;
```

```
 * mysql+pymysql://root:***@localhost:3306/Carbon_Trading_DB
5 rows affected.
```

Out[33]:

Quarter	Transaction_Type	Count
2024-Q1	Buy	207
2024-Q1	Hold	224
2024-Q1	Sell	147
2024-Q2	Buy	191
2024-Q2	Hold	229

Python Business Query

In [34]:
```
df["Quarter"] = df["Date"].dt.to_period("Q").astype(str)

q_trade = (df.groupby(["Quarter", "Transaction_Type"])
           .size().reset_index(name="Count")
           .sort_values(["Quarter", "Transaction_Type"]) )

q_trade.head()
```

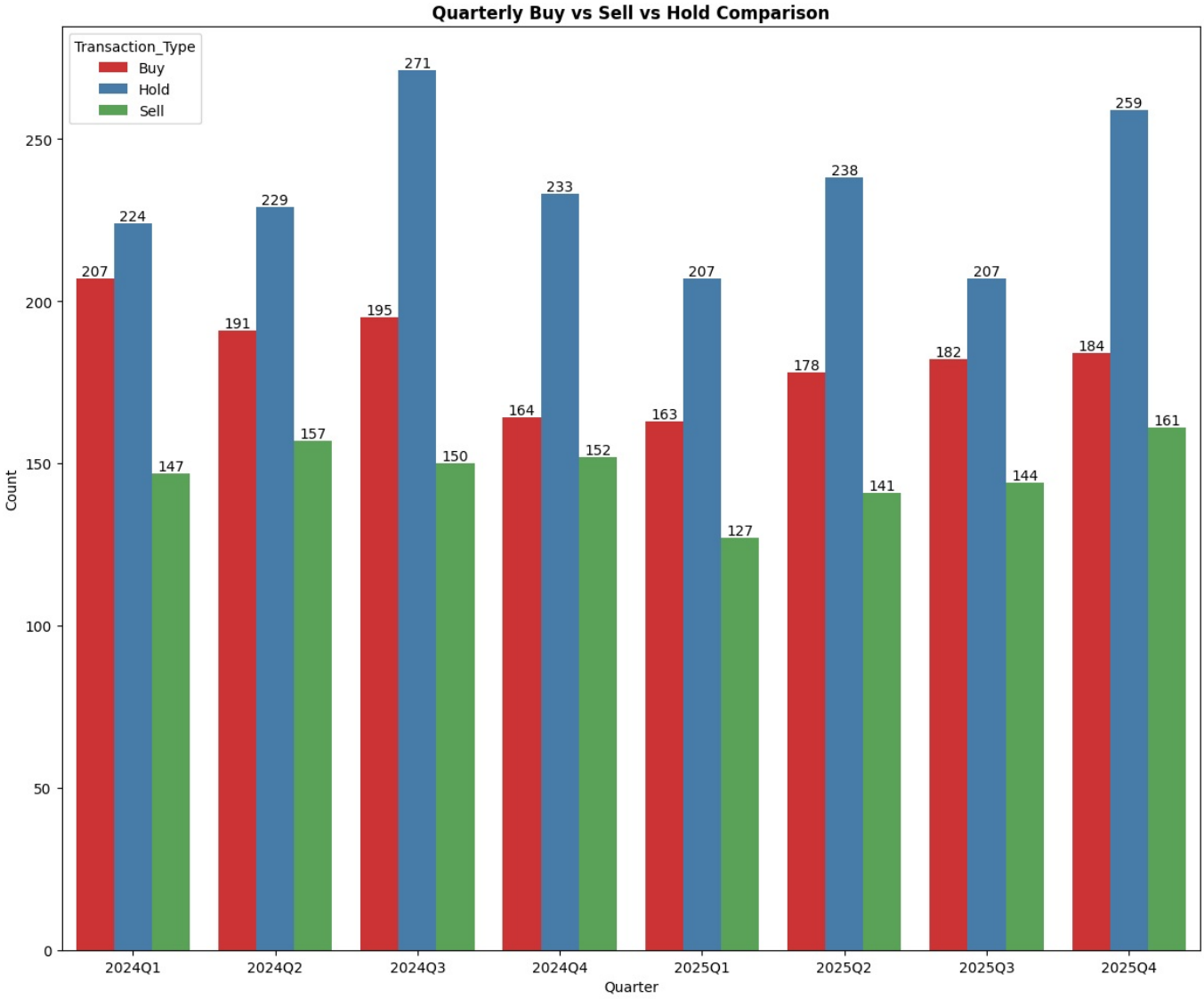
Out[34]:

	Quarter	Transaction_Type	Count
0	2024Q1	Buy	207
1	2024Q1	Hold	224
2	2024Q1	Sell	147
3	2024Q2	Buy	191
4	2024Q2	Hold	229

Visualization of Query

In [35]:
```
plt.figure(figsize=(12,10))
```

```
ax = sns.barplot(
    data=q_trade,
    x="Quarter",
    y="Count",
    hue="Transaction_Type",
    palette="Set1"
)

# Add data labels
for container in ax.containers:
    ax.bar_label(container, fontsize=10)

plt.title("Quarterly Buy vs Sell vs Hold Comparison", weight="bold")
plt.tight_layout()
plt.show()
```



Q9 — Carbon Price Trend

SQL Business Query

In [36]:
```
%%sql
SELECT
    Date,
    ROUND(AVG(Carbon_Price_USD_per_t), 2) AS Avg_Price
FROM sql_eda_carbon_clean
GROUP BY Date
ORDER BY Date
LIMIT 5;
```

```
 * mysql+pymysql://root:***@localhost:3306/Carbon_Trading_DB
5 rows affected.
```

Out[36]:

Date	Avg_Price
2024-01-01 00:00:00	28.25
2024-01-02 00:00:00	28.16
2024-01-03 00:00:00	28.53
2024-01-04 00:00:00	27.16
2024-01-05 00:00:00	27.93

Python Business Query

In [37]:

```
q9 = (
    df.groupby("Date")["Carbon_Price_Usd_Per_T"]
      .mean()
      .reset_index(name="Avg_Price")
)

q9.round(2).head()
```

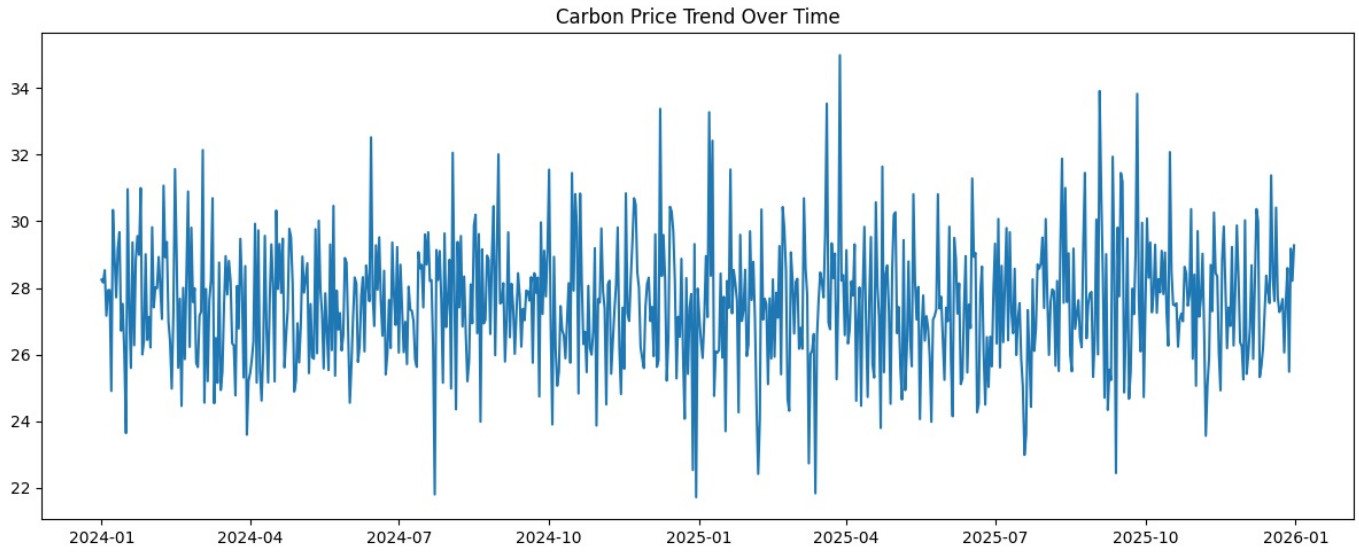
Out[37]:

	Date	Avg_Price
0	2024-01-01	28.25
1	2024-01-02	28.16
2	2024-01-03	28.53
3	2024-01-04	27.16
4	2024-01-05	27.93

Visualization of Query

In [38]:

```
plt.figure(figsize=(12,5))
plt.plot(q9["Date"], q9["Avg_Price"])
plt.title("Carbon Price Trend Over Time")
plt.tight_layout()
plt.show()
```



Q10 — Industry Compliance Cost

SQL Business Query

In [39]:

```
%%sql
SELECT
    Industry_Type,
    ROUND(AVG(Compliance_Cost_USD), 2) AS Avg_Compliance_Cost
FROM sql_eda_carbon_clean
GROUP BY Industry_Type
ORDER BY Avg_Compliance_Cost DESC
LIMIT 5;
```

```
 * mysql+pymysql://root:***@localhost:3306/Carbon_Trading_DB
4 rows affected.
```

Out[39]:

Industry_Type	Avg_Compliance_Cost
Steel	12615.45
Energy	12593.11
Cement	12497.93
Manufacturing	12323.62

Python Business Query

In [40]:

```
q10 = (
    df.groupby("Industry_Type")["Compliance_Cost_Usd"]
      .mean()
      .reset_index(name="Avg_Compliance_Cost")\
      .sort_values(by="Avg_Compliance_Cost", ascending=False)
)

q10.round(2)
```

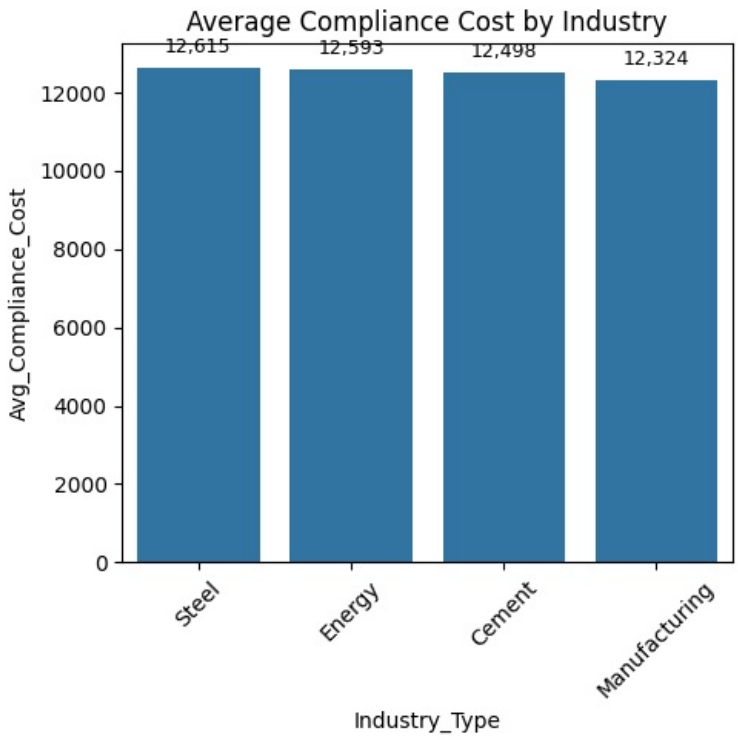
Out[40]:

	Industry_Type	Avg_Compliance_Cost
3	Steel	12615.45
1	Energy	12593.11
0	Cement	12497.93
2	Manufacturing	12323.62

Visualization of Query

In [41]:

```
plt.figure(figsize=(5,5))
ax = sns.barplot(data=q10, x="Industry_Type", y="Avg_Compliance_Cost")
add_labels(ax)
plt.xticks(rotation=45)
plt.title("Average Compliance Cost by Industry")
plt.tight_layout()
plt.show()
```



Q11 — Energy vs Emissions Relationship

SQL Business Query

In [42]:

```
%%sql
SELECT
    Energy_Demand_MWh,
    Emission_Produced_tCO2
FROM sql_eda_carbon_clean
LIMIT 5;
```

```
 * mysql+pymysql://root:***@localhost:3306/Carbon_Trading_DB
5 rows affected.
```

Out[42]:

Energy_Demand_MWh	Emission_Produced_tCO2
2523.15	1532.65
1747.79	1205.54
977.27	731.4
1747.79	1396.17
1747.79	1485.77

Python Business Query

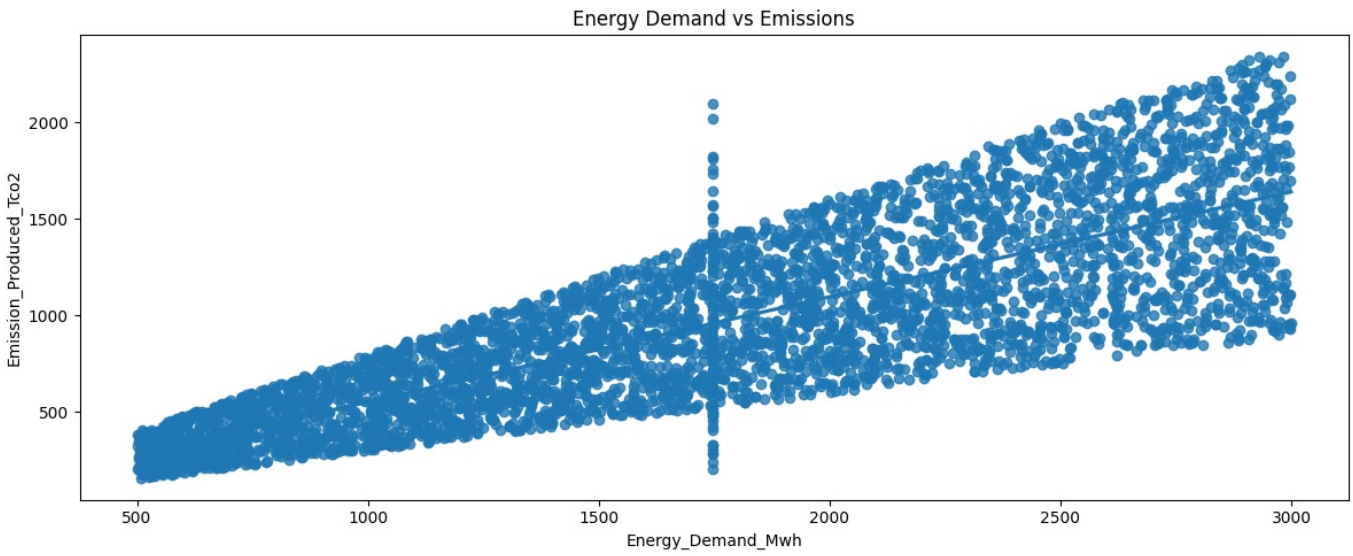
In [43]:
```
q11 = df[["Energy_Demand_Mwh", "Emission_Produced_Tco2"]]
q11.head()
```

Out[43]:

	Energy_Demand_Mwh	Emission_Produced_Tco2
0	2523.15	1532.65
1	1747.79	1205.54
2	977.27	731.40
3	1747.79	1396.17
4	1747.79	1485.77

Visualization of Query

In [44]:
```
plt.figure(figsize=(12,5))
sns.regplot(
    data=df,
    x="Energy_Demand_Mwh",
    y="Emission_Produced_Tco2"
)
plt.title("Energy Demand vs Emissions")
plt.tight_layout()
plt.show()
```



Q12 — Top Carbon Cost Saving Companies

SQL Business Query

In [45]:
```
%%sql
SELECT
    Company_ID,
    ROUND(SUM(Carbon_Cost_Savings_USD), 2) AS Savings
FROM sql_eda_carbon_clean
GROUP BY Company_ID
ORDER BY Savings DESC
LIMIT 5;
```

```
 * mysql+pymysql://root:***@localhost:3306/Carbon_Trading_DB
5 rows affected.
```

Out[45]:

Company_ID	Savings
C054	110927.93
C006	105558.21
C052	99899.36
C078	99621.68
C035	99237.2

Python Business Query

In [46]:

```
q12 = (
    df.groupby("Company_Id")["Carbon_Cost_Savings_Usd"]
      .sum()
      .reset_index(name="Savings")
      .sort_values(by="Savings", ascending=False)
      .head(10)
)

q12
```

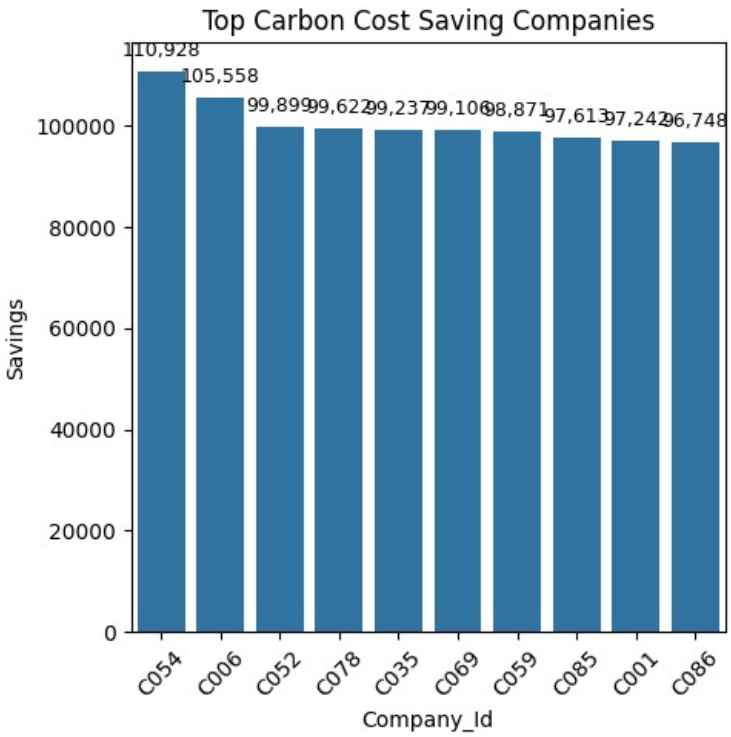
Out[46]:

	Company_Id	Savings
53	C054	110927.93
5	C006	105558.21
51	C052	99899.36
77	C078	99621.68
34	C035	99237.20
68	C069	99105.74
58	C059	98871.43
84	C085	97612.69
0	C001	97241.89
85	C086	96748.30

Visualization of Query

In [47]:

```
plt.figure(figsize=(5,5))
ax = sns.barplot(data=q12, x="Company_Id", y="Savings")
add_labels(ax)
plt.xticks(rotation=45)
plt.title("Top Carbon Cost Saving Companies")
plt.tight_layout()
plt.show()
```



In [ ]: